Human Activity Recognition with PCA

Machine Learning Lab 11 (Open Ended Lab) Report

Name: Abdul Azeem **Section:** BAI-5A **Roll No:** 24i-2013

Introduction and Tools

The task is to recognize six activities (walking, walking upstairs, walking downstairs, sitting, standing, laying) from smartphone sensor data, for a wearable with limited memory and battery. In the lab we have studied Linear Regression, Logistic Regression, PCA, KNN, Decision Tree, Random Forest, Bagging and Boosting. For this task I used PCA, KNN, Decision Tree and Random Forest. I wrote the notebook myself and ran it in **Google Colab**, which helped me a lot in writing and debugging my code, because I could run and fix one cell at a time. The notebook was not generated by Claude.

Step 1: Data Exploration

- The data has 7352 training and 2947 test samples, 561 sensor features and 6 activity classes.
- The classes are fairly balanced (986 to 1407 samples, Figure 1), so accuracy is a fair metric.
- Box plots (Figures 2 and 3) show that tGravityAcc-mean()-X separates LAYING clearly from all other activities, while tBodyAcc-mean()-X is almost the same for every activity and separates them poorly.
- Many features are redundant: 56 of the first 100 features have correlation above 0.9 with another feature.
- The data comes from 30 people. Train has 21 subjects and test has 9 different subjects with no overlap. I kept this split, because it tests the model on new users.

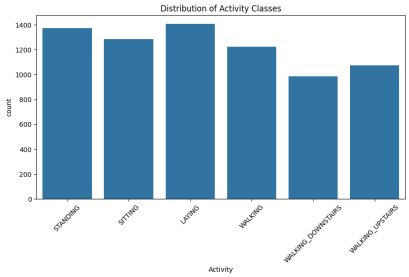


Figure 1: Class distribution

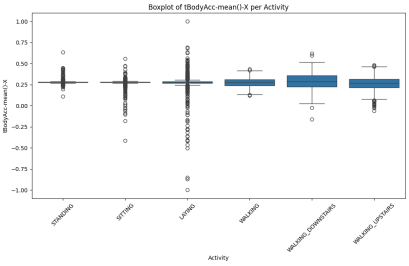


Figure 2: tBodyAcc-mean()-X per activity

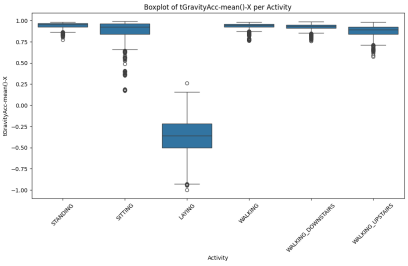


Figure 3: tGravityAcc-mean()-X per activity

Step 2: Data Preprocessing

- There are no missing values in the train or test data.
- I used **StandardScaler** (fitted on train data only to avoid leakage), because PCA and KNN depend on distances and variance.
- PCA needs **102 components** to keep 95% of the variance (Figure 4), so features drop from 561 to 102.
- In the 2D PCA plot (Figure 5), moving activities are separated from static ones, but sitting, standing and laying overlap, and the walking types overlap partly.

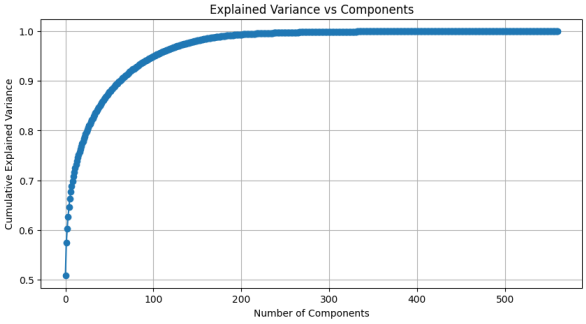


Figure 4: Explained variance vs components

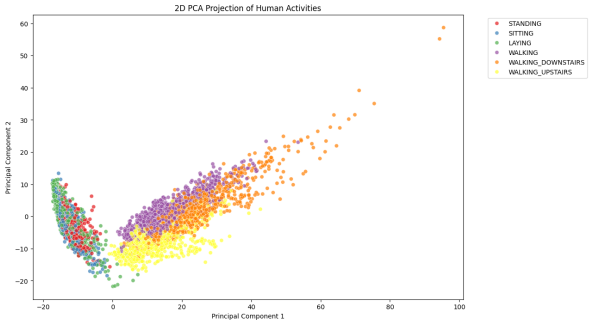


Figure 5: First two principal components

Step 3: Build Classification Models

I trained KNN (k = 5), Decision Tree (depth 10) and Random Forest (100 trees) on the full features and on 102 PCA components. Times are in seconds (single run on Colab, prediction time is for the whole test set).

Model	Features	Accuracy	Train time	Predict time
KNN	Full (561)	88.36%	0.085	3.185
KNN	PCA (102)	87.68%	0.089	0.841
Decision Tree	Full (561)	86.66%	7.984	0.005
Decision Tree	PCA (102)	76.25%	1.341	0.003
Random Forest	Full (561)	92.60%	22.376	0.052
Random Forest	PCA (102)	88.16%	12.482	0.054

Step 4: Evaluate Models

I used accuracy for all models, and for the best model (Random Forest on full features) I also used the per class report and a confusion matrix (Figure 6), because accuracy alone hides which classes are mixed up. Its F1 scores are: LAYING 1.00, SITTING 0.90, STANDING 0.91, WALKING 0.92, WALKING_DOWNSTAIRS 0.91, WALKING_UPSTAIRS 0.90. Overall accuracy is 92.6%. I tested only one PCA setting (102 components), so I did not plot accuracy and time against many component counts. I use 102 components as the trade-off, because it keeps 95% of the variance and cuts features by about 82%.

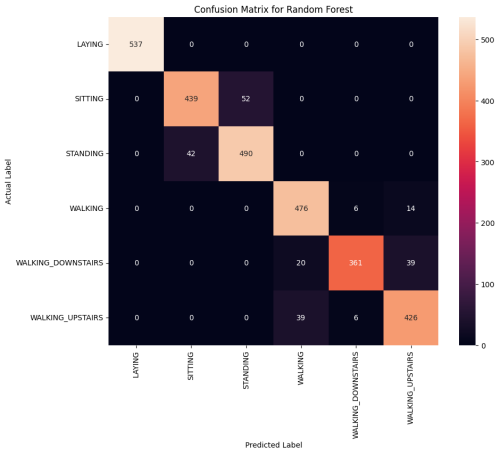


Figure 6: Random Forest confusion matrix

Step 5: Interpretation and Insights

- **Easy:** LAYING is recognized perfectly (537 of 537), because the gravity direction changes completely when the body is flat.
- **Confused:** SITTING and STANDING are mixed up (52 and 42 errors), because the phone is at the waist and the body is almost still in both. The walking types are also mixed (for example 39 upstairs samples predicted as walking), because their motion is similar.
- **Model comparison:** Random Forest is the most accurate but trains slowest. KNN trains instantly and is easy to explain, but predicts slowly and must store all training data. Decision Tree is the easiest to explain and fastest at prediction, but least accurate.
- **PCA effect:** KNN became about 4 times faster at prediction with only 0.7% accuracy loss. Random Forest trained faster (22.4 s to 12.5 s) but lost 4.4% accuracy, and Decision Tree lost 10.4%. PCA keeps the large variance that separates static and moving activities, but loses small details that separate similar classes.
- **Chosen model: Random Forest.** It has the highest accuracy (92.6%) and F1 of 0.90 or more for every class. As a bagging type ensemble it is more stable than one Decision Tree and beats KNN. Prediction is very fast (0.05 s for 2947 samples) and it does not store training data. Its slow training happens once on a computer. If memory is very tight, Random Forest on PCA features (88.2%) is a lighter option.
- **Limitations and future work:** only one PCA setting was tested, and Logistic Regression, Bagging and Boosting were not run in the notebook. Timings come from one run. New users or different sensor placement may reduce accuracy. In future I would test several component counts, add Logistic Regression and Boosting, and reduce the number of trees to save memory.